

RAPPORT DE PROJET ALGO 4 : AEROPORT

Par TOURE Mamadou & SECK Mamadou

1. Présentation Générale

Ce projet s'inscrit dans le cadre du module d'Algorithmique et Structures de Données. L'objectif est de simuler l'organisation d'un aéroport à l'aide d'un programme en C. Le système permet d'enregistrer des vols, d'organiser leur décollage en fonction des priorités d'urgence, et d'offrir une fonctionnalité d'annulation (Undo) pour corriger les erreurs de saisie.

Afin de garantir une structure claire et un code lisible, le projet a été découpé de manière modulaire en quatre modules (vols, filePiste, pile et main).

2. Structures de Données Utilisées

Le projet s'appuie sur trois structures de données fondamentales :

- **Liste Chaînée Doublement Chaînée (Registre des vols) :**

Chaque maillon pointe vers le vol précédent (pred) et le vol suivant (suiv). Cette structure permet une insertion triée des vols selon l'heure de départ. Elle facilite également l'affichage dans les deux sens (chronologique et inverse) et rend la suppression d'un élément simple sans parcourir toute la liste depuis le début si l'on a le pointeur courant.

- **File d'Attente Prioritaire (Piste d'envol) :**

Définie avec un pointeur sur la tête et la queue, elle gère l'ordre de passage des avions. Un champ urgence (0 ou 1) permet de traiter les imprévus : les vols urgents sont placés en tête de file (après les autres vols urgents éventuels), respectant le principe FIFO pour les vols de même priorité.

- **Pile Borne à 10 Éléments (Historique des actions) :**

Utilisée selon le principe LIFO, elle enregistre les opérations effectuées (AJOUT_VOL, SUPPRESSION_VOL, DECOLLAGE). Pour respecter le sujet, la pile est limitée aux 10 dernières actions : lorsque la taille atteint 10, l'élément le plus ancien situé tout en bas de la pile est supprimé.

3. Choix de Conception

- **Modularité Simple :** Le choix de diviser le code en plusieurs fichiers .h et .c évite d'avoir un fichier source trop volumineux. La compilation séparée permet d'isoler le comportement de chaque structure de données.
- **Sauvegarde des Vols Supprimés :** Dans la fonction supprimerVol, le paramètre volSupprime permet de conserver les informations du vol effacé pour pouvoir reconstruire la liste lors d'une annulation (Undo).

- **Pointeurs et Allocation Dynamique** : L'utilisation de pointeurs et de la fonction malloc permet de gérer la mémoire au fur et à mesure des besoins sans fixer une limite stricte au nombre total de vols du registre.

4. Difficultés Rencontrées

- **Gestion de la suppression en bas de Pile** : La contrainte d'avoir une pile bornée à 10 éléments a nécessité la création d'une fonction supprimerFond. Retirer l'élément au fond de la pile en ne manipulant que des opérations de dépilage/empilage requiert l'utilisation d'une pile auxiliaire temporaire pour ne pas inverser l'ordre initial.
- **Synchronisation du Registre et de la File** : Lors de la suppression d'un vol ou lors de l'annulation d'une action, il a fallu s'assurer que le vol était également retiré de la file de la piste si celui-ci y avait été placé, afin d'éviter d'avoir des références vers des données supprimées.